

PlantSketch: Creating Realistic and Customisable Plant Ecosystems

William Marketos
Department of Computer Science
MRKWIL007@myuct.ac.za

Ella Thorburn
Department of Computer Science
THRELL002@myuct.ac.za

Jacob Erasmus
Department of Computer Science
ERSJAC011@myuct.ac.za

29 September 2025

Abstract

This report covers the creation of a realistic and customisable forest ecosystem simulator through statistical analysis. It will allow our client James Gain as well as other stake-holders such as ecosystem researchers, biologists, horticulturalists, botanists, game environment developers and private users to automatically and speedily generate realistic and diverse plant ecosystems on any three dimensional terrain map. This map contains data about the abiotic factors that would influence the growth of a plant, namely sunlight, moisture levels, average temperature, and the slope of the terrain. The simulator allows for near-instantaneous generation of hyper-realistic ecosystems and allows the user to customise any and all variables they can think of through both global and local means.

1. Introduction

Plant ecosystems are complicated environments that provide an interesting challenge for computer scientists. The number of factors that influence the state of a given ecosystem is immense, and finding a way to consider all of these factors when artificially generating an ecosystem is a complex but necessary problem to tackle. Interactive artificial representations of ecosystems can be incredibly helpful tools for conservation, forestry management, risk assessment and computer games and it is in the interest of all users to make these representations as accurate and performant as possible.

The purpose of our project is to provide our own solution to this challenge, providing our client with a program that uses statistical sampling, files containing abiotic environmental factors, and a viability and logistic growth calculator to generate a realistic plant ecosystem. The ecosystem is displayed visually for the user via a Graphical User Interface, and provides features to interact with the environment via a “paint brush” tool that adjusts abiotic factors on demand and then recalculates and displays the resulting forest. The user also has the ability to remove and add certain species.

The challenge of providing a performant GUI proved immense, and we decreased the intimidating nature of this task by following an incremental development strategy. Building the system component by component allowed us to check integration of modules and validate results of calculations before proceeding with other components. We used console logging at the beginning of development to verify correctness of numbers for viabilities and densities at given points and also visualized our reading of all abiotic factors from the file manager in order to verify they appeared as expected. The use of unit tests also proved invaluable to ensuring no new features or logic broke existing behaviour.

Two academic papers were consulted when considering our approach to this challenge, namely “DeadWood: Including Disturbance and Decay in the Depiction of Digital Nature” (J. Gain et al.) and “Data-Driven Authoring of Large-scale Ecosystems”(J. Gain et al.). These provided valuable guidance on professional practices and approaches and proved insightful reading for the project.

We used an agile approach to develop this product, and regular meetings in the formative stages provided suitable feedback to questions that arose throughout the development cycle and made our approach effectively follow an incremental development method. Our planning of the project was comprehensive, and the utilization of a Gantt chart proved invaluable in guiding our next steps and meeting deadlines. We also held in person meetings which sped up our productivity through collaborative coding and problem-solving. Another tool that proved invaluable to us was GitLab, which allowed us to use version control to collaboratively and iteratively develop our product.

Presenting a prototype to our client also proved invaluable for guiding our progress, and proved the value of using an agile approach, as our priorities of which features to add and fixes to make adjusted after the presentation of our prototype. We were able to identify performance and logic problems within our program early on in the development cycle because of this.

2. Requirements Captured

2.1. Functional Requirements

Input data handling: The application must load in abiotic maps as input for sunlight, elevation, moisture, temperature and age from a folder of the user's choosing. The application must be able to handle the reading of custom .txt, .elv, .png and .pdb files. The system should validate that all input files are consistent in dimensions and grid spacing.

Sampling and plant placement: The application must generate a random pink noise distribution with a target number of sample points chosen by the user. For each point, the application must: Calculate the viability for each species at that point using the viability calculator, abiotic data at that point, and the viability parameters of that plant. Use a roulette wheel selection method to randomly allocate a species to that point. Calculate and assign the growth parameters to that plant. Place the plant at that point.

2D Rendering: The application must then generate a simplified two-dimensional representation of the forest overlaid on top of each of the abiotic maps so the client can see relations between plants and abiotic factors.

User interaction: There is an option for the user to choose between 'test', 'run', and '3D' mode. 'test' is a 2x2 grid in which the user can quickly and easily tailor many variables and get a close-up view of how each variable affects the plant growth. 'run' is the main programme in which the user must be able to upload their own files. The application must allow the user to manipulate all plant and abiotic variables using a mix of global and localised changes through the 'species parameter panel' and the brush interface respectively. The user must also have the ability to undo and redo changes in case they did not like a change made, and the option to save their progress.

Output handling: The user must have the ability to output a .pdb file at any time in order to save their progress. The format of the output file is a custom .pdb file that the user can then use to upload to either EcoViz or the programme's in-built 3d generator.

3D generation: Using the generated .pdb file the programme must associate it with its corresponding .elv file and allow the user to view their generated forests in 3D.

2.2. Non-Functional Requirements

Performance:

Efficiency: The system should load and render the visualization of the inputted data and display updated changes in a way that provides a reasonably smooth experience for the user. Efficiently calculating Pink Noise Sample points, Viabilities and Heights with Canopy Radiuses is a priority to make the visual experience of the system pleasing. *Scalability:* The system should be able to handle larger sized data inputs and larger brush sizes.

Usability:

User Input Clarity: The functionality of the modifiable parameters of the user interface must be clear and intuitive for the user, with good prompts and expected behaviour for sliders and buttons. *User Interface Simplicity:* The interface should not contain unnecessary clutter and functionality and visuals must be separated into separate view screens and sections of the parameter adjustment pane *Error Handling:* The GUI must display relevant error messages to the user to guide their experience.

Maintainability:

Code Modularity: The systems should be designed with modules that are easy to be used by other modules and be structured in such a way that internal changes still allow usage of methods and constructors as expected, this makes it easy to add functionality for different views and optimizing components that the update system uses *Documentation:* The system should include solid documentation that makes it easy for a new user/developer to understand.

Graphical User Interface Quality:

The graphical user interface should be visually appealing to the user.

2.3. Significant Classes and Methods in Programme

A. Interface createTabs(): Build visualization tabs (Env, Pink Noise, Forest).

createForestOnMapTab(map, type): Overlay forest on env maps with zoom.

executeSimulation(choice, reSim, path): Main controller for running sims.

readGridEditors(): Load edited 2x2 grids (Test Mode).

buildParameterPanelTest(): Test Mode controls.

buildParameterPanelRun(): Run Mode controls + brush tools.v

B. SimulationEngine processPlantPlacement(): Place plants via viability + roulette selection.

calculateViabilityTable(): Build cumulative viability tables per location.

C. ForestOnMapView draw(): Render heatmap + plant overlay.

drawMap(): Draw env heatmap.

drawForest(): Overlay species plants.

enable/disableBrushRemovalMode(): Plant removal via brush.

D. Terrain getSlope(x, y): Return cached slope (degrees).

calculateSlopeGrid(): Precompute slopes for O(1) lookup.

calculateSlopeAt(x, y): Compute slope via elevation gradients.

E. ViabilityCalculator viability(species, x, y): Return min viability across slope, temp, sun, moisture.

calculateViability(r, c, v): Core formula $(1 + a) * e^{(d/r)^4 \cdot 0.5 * \ln(0.2)} - a$.

F. RouletteWheelSelector selectSpecies(table, count): Weighted random selection by cumulative viability.

RouletteWheelSelector(sum): Constructor picks random threshold.

G. PinkNoiseSampler generateSamples(count, dimX, dimY, spacing): Pink noise sampling for plant distribution.

H. MainApp start(stage): JavaFX entry point, opens mode selection.

showModeSelection(stage): Buttons for Test, Run, 3D view.

centerStage(stage): Auto-center window.

I. GridEditor setValues(values): Populate grid.

getValues(): Extract validated values.

isEdited(): Detect modified values.

J. ViewTransform ViewTransform(dimX, dimY, spacing): Grid/pixel scaling.

metersToPixelX/Y(): Real $\rightarrow$ pixel coords.

pixelToGridX/Y(): Pixel $\rightarrow$ grid coords.

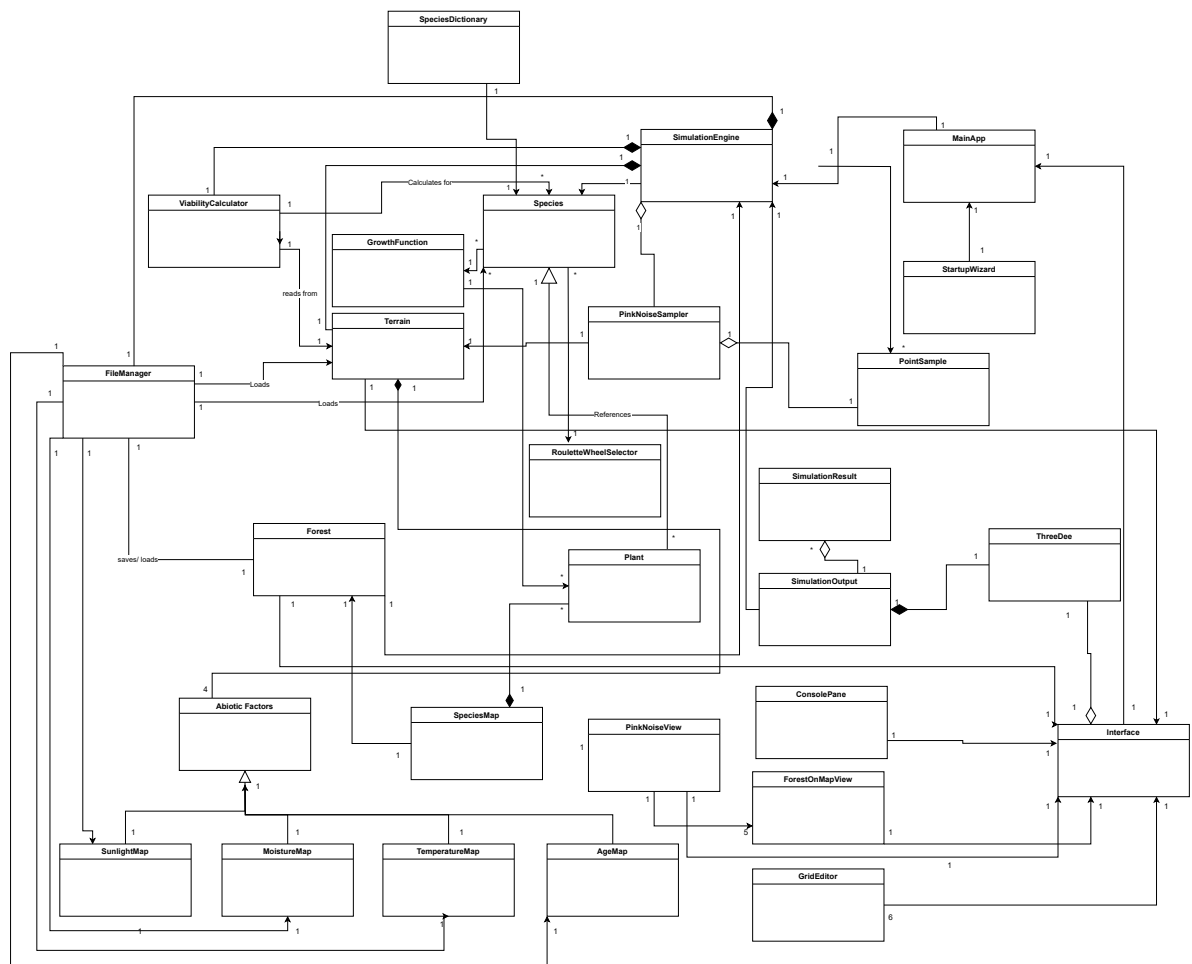


Figure 1: Class Diagram for our Application

2.4. Use Cases

2.4.1. Use Case 1: User uses Test Mode

Actor: User

The User starts the program. The system prompts the User to choose between 'test', 'run' and '3D' mode. They choose 'test' mode. The user then chooses from one of the preset options or a random option that randomly generates all abiotic values. The user can edit the abiotic and species data within the GUI as much as they want, with options for undo and redoing mistakes and viewing previous instances.

2.4.2. Use Case 2: User uses Run Mode

Actor: User

The User starts the program. The system prompts the User to choose between 'test', 'run' and '3D' mode. They choose 'run' mode. The system prompts the User to choose to use the default inputs or input their own files. *Alternative Paths: (1) If the User selects to use the default inputs, the system prompts the User to choose a specified grid size after which the system loads the files from the corresponding folder. (2) If the User selects to input their own files, they upload the files which are of correct format and mapping to the system. The system loads the uploaded files. (2a) Should the User try to upload an incorrectly formatted file or mapping, the system will reject and re-prompt the User to choose to use the default inputs or input their own files. End of Alternative Paths*. The system then generates a realistic sampling. The system outputs the results to a file and graphically renders a 2D visualisation interface. The user can edit the abiotic and species data within the GUI as much as they want through either local changes using the 'brush' tool or through global changes through the 'species parameters' tool. There are also, options for undoing and redoing mistakes and viewing previous instances. When they are happy with their results they can save the forest to a .pdb file with a name of their choosing to the folder the data came from.

2.4.3. Use Case 3: User uses 3D Mode

Actor: User

The User starts the program. The system prompts the User to choose between 'test', 'run' and '3D' mode. They choose '3D' mode. The user is then asked to pick a folder from which they wish to view the .pdb and .elv files contained in a simplified 3-dimensional representation. Once the file is chosen the forest and terrain are rendered in 3D and the user can zoom in, scroll and admire the beautiful forest they made.

3. Implementation

Our project's approach is well established within the field and has three main components:

- a data field based off of a coordinate system, which provides a simple but robust way to map biotic and abiotic information using 2d arrays.
- A sampling method that produces point samples based off of a Pink Noise distribution, which provides a satisfactory approximation of natural forest distributions.
- A calculation method, which determines plant placements and characteristics.

These components are integral to the programme, which generates the final output by executing as follows:

- The ecosystem abiotic factors are loaded and encapsulated by a terrain object which holds the pointers to the 2d arrays and the slope at each point is calculated and cached
- The pink noise samples are mapped to corresponding coordinates of the terrain
- For each of these points, the viability of each species is calculated

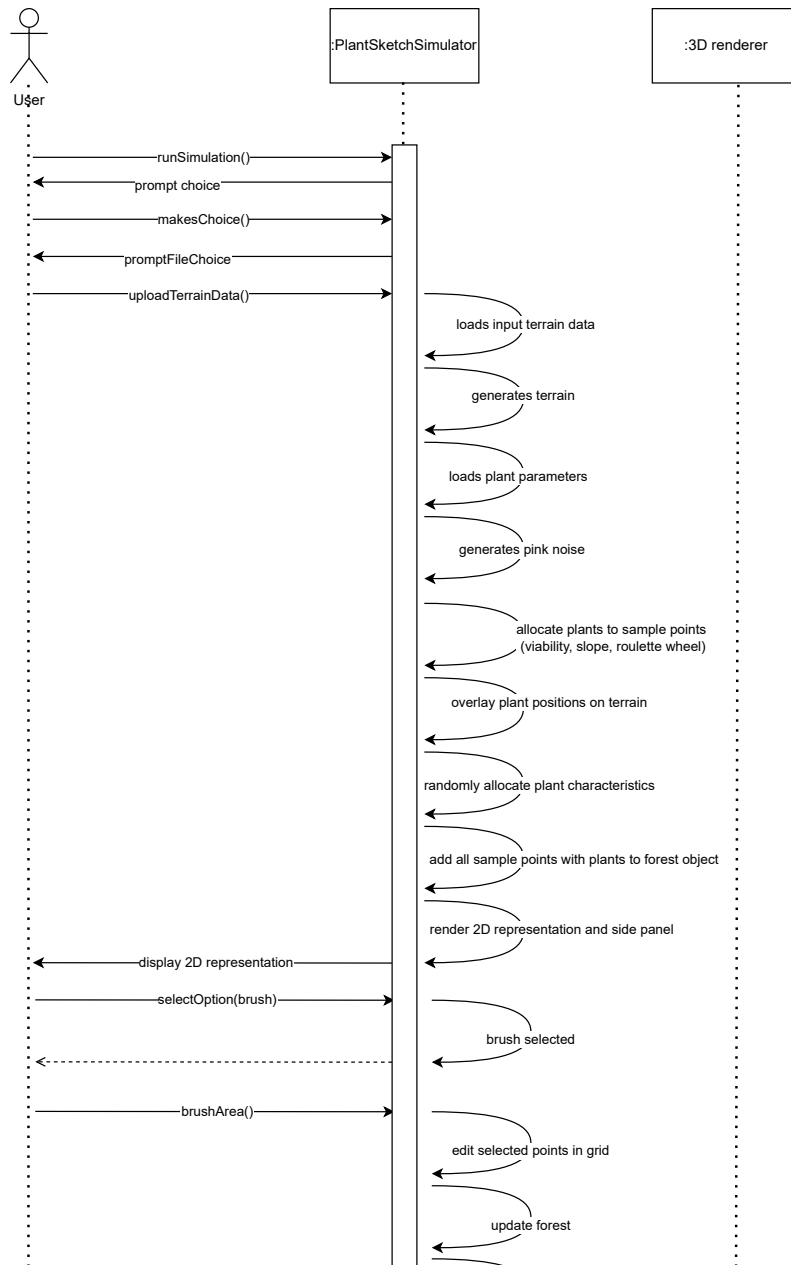


Figure 2: Sequence Diagram for our Application I

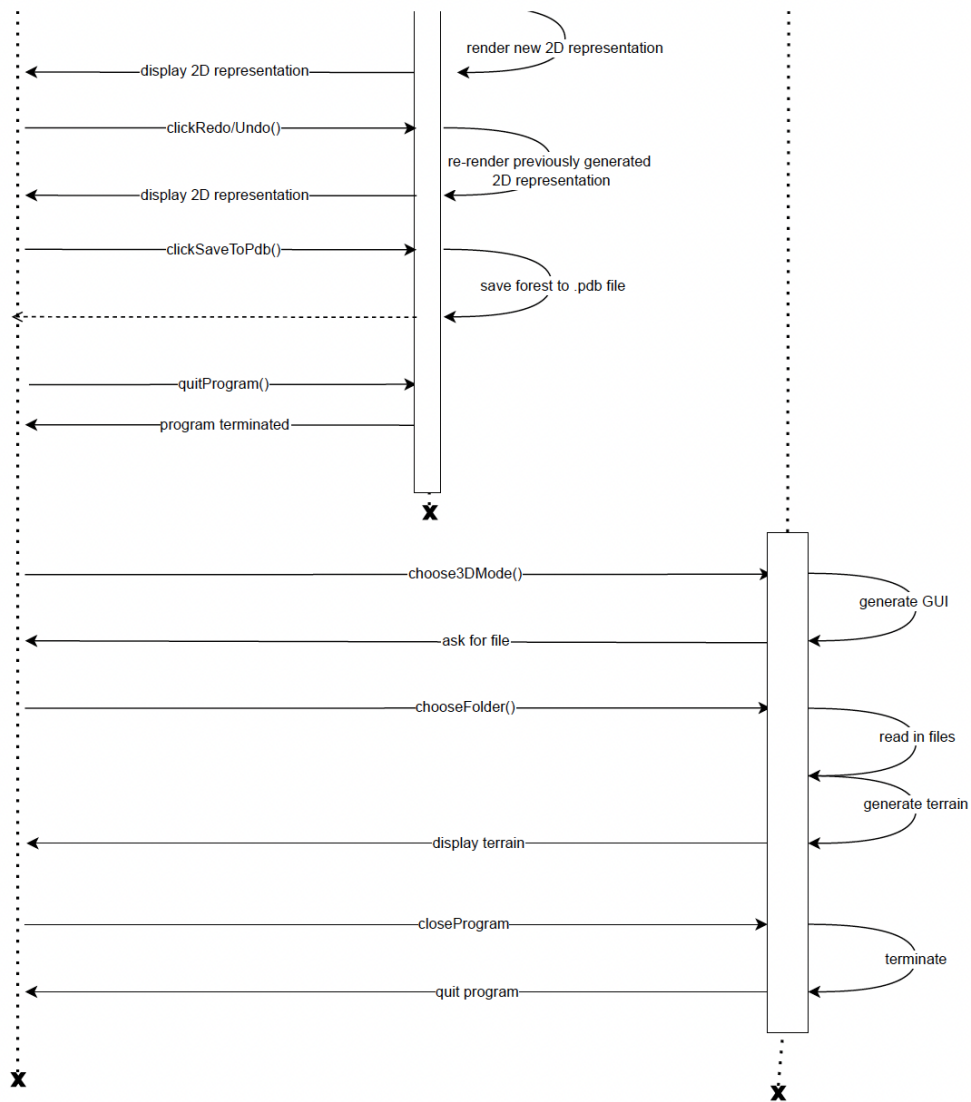


Figure 3: Sequence Diagram for our Application II

- The roulette wheel selector chooses a species for placement
- The logistic growth function calculates the characteristics of the placed plant
- The GUI utilizes the calculated Forest to show the user a visual representation
- The program generates a .pdb file which can be sent to ecoviz or our in-built visualizer for 3d visualization

4. Program Validation and Verification

In order to assert the correctness of our program, we tested bottom-up, testing lower-level modules first before asserting expected output by larger modules. We also tested incrementally, integrating modules one by one after testing them individually. This allowed us to catch errors and unexpected behaviour at all stages of the SDLC. (see Table 1).

Table 1: Summary Testing Plan.

Process	Technique
1. Class Testing: test individual methods and behaviour of classes	Asserting known hardcoded values against method calculations, Asserting object instance as expected
2. Integration Testing: test the interaction of sets of classes	Calling methods of classes that use other classes methods and instance variables
3. Validation Testing: test whether customer requirements are satisfied	GUI interaction, examining response and features matches expected
4. System Testing: test the behaviour of the system as part of a larger environment	Stress and Performance tests

1. **Class Testing** We used unit testing and started by testing our smaller classes and methods that have no dependencies, such as the ViabilityCalculator, HeightGenerator, PinkNoiseSampler and RouletteWheelSelector. All of these can be tested using dummy data for expected behaviour, determined by doing manual calculation. Manually inspecting the graphical output for data using JavaFX will also be used to assert that the PinkNoiseSampler, global changes, and Brush Tool are functioning as expected. We will fix random seeds for the PinkNoiseSampler and RouletteWheelSelector to allow replicable results.

2. Integration Testing

3. System Testing

Stress Tests: This was performed by interacting with the graphical user interface with variable inputs. The approach was to examine the performance of the system under various loads and survey visual feedback as well the output from our PerformanceTimer. We found that the greatest slow down of our interface was caused by the initial rendering of the simulation, specifically the file reader. The impact of this slow down was greatly increased as the grid size inputted became larger. We were however satisfied that the speed of visual feedback for the brush tool remained reasonably consistent across the grid, showing effective isolation of calculations and renderings for changes using the brush. The number of samples placed also showed to contribute significantly to increasing the time it took to execute the simulation. Requesting too many changes to the interface too quickly also resulted in significant stress to the system and poor responsiveness, as expected.

Performance Testing: The performance tests were formulated to measure and compare the execution time and change application times for the programme before and after performing optimizations that cached slope calculations and colour calculations for drawing on the GUI. The tests were

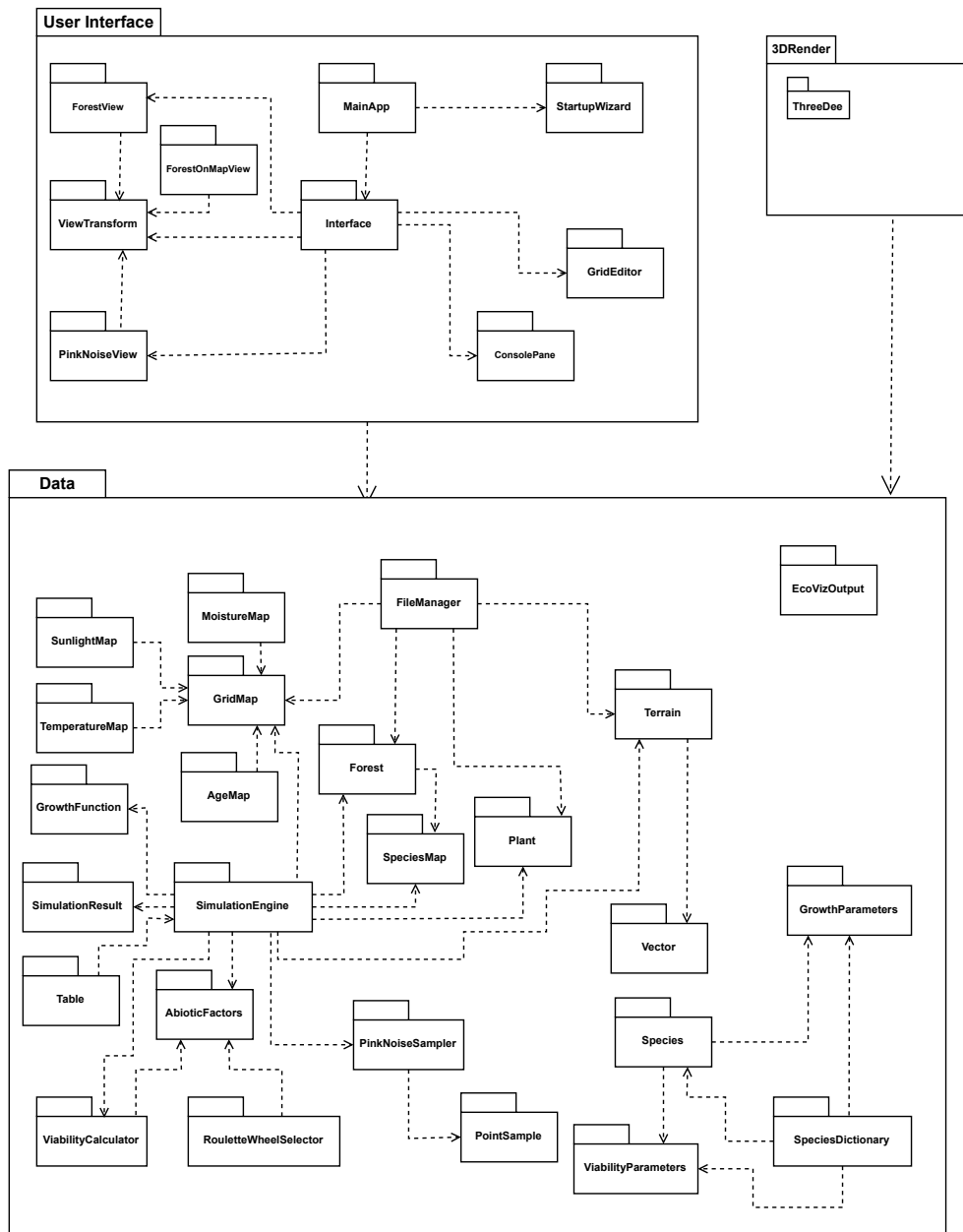


Figure 4: Architecture Diagram for our Application

Table 2: A table of tests.

Integrating Classes	Tested Interaction	Test Case Description	Test Methods Used
Viability Calculator and Species and Terrain	Viability calculation for a species at a given point on the terrain	Calculating expected high/low values for viability at optimal and suboptimal conditions respectively	testBoxwoodViabilityOptimalConditions() testSilverFirViabilitySuboptimalConditions()
Pink Noise Sampler and Point Sample	Pink Noise Generator creating a searchable List of Sample Points	Ensure that the pink noise samples have the same point sample value location when there is a deterministic seed	deterministicWithSeed()
RouletteWheelSelector and ViabilityCalculator	Species selection based on viability weights	Test probabilistic species selection using calculated viability values as weights. Ensure proper handling of edge cases.	testRouletteWheelSelectionSingleViableSpecies()
FileManager and Environmental Maps	Environmental data loading and 2D grid creation	Test complete environmental data pipeline from file loading to grid creation. Validate data integrity across different file formats.	testElevationFileLoading() testTextFileLoading()
GrowthFunction and Species	Growth calculation using species-specific parameters	Test growth calculations using real species parameters. Validate growth constraints and age-dependent behavior.	heightGrowsWithAge OpenAllometry() respectsClosedAllometryMax()

conducted by strategically adding a recording point for the start and end of a code snippet using our performanceTimer class. We assessed the impact of these changes using data from our performance timer pulled before and after implementing the data optimization. Speedup was calculated by dividing the original calculated time by final calculated time, where time is measured in total milliseconds taken to execute a simulation, represented by the graph below.

4. **Validation Testing:** Validation testing involved extensive interaction with the ui and tests to ensure that the requirements were met and functioned as expected.

Input data handling

Requirement: The application must accurately load in abiotic maps as input for sunlight, elevation, moisture, temperature and age from a folder of the user's choosing *Validation:* Visual rendering of all abiotic grids including age and elevation show that they have been read in accurately as expected

Sampling and plant placement

Requirement: The application must generate a random pink noise distribution with a target number of sample points chosen by the user and perform all necessary calculations to pick a plant at that point and output it visually, with size based off of canopy radius. *Validation:* Plants seem to populate the environment in a way that intuitively makes sense based off of the underlying abiotics that are able to be visualized and unit testing validates values

2D Rendering

Requirement: The application must then generate a simplified two-dimensional representation of the

forest overlayed on top of each of the abiotic maps so the client can see relations between plants and abiotic factors. *Validation:* Fulfilled to a very high standard by the graphical user interface

User interaction

Requirement: Program allows global species parameter changes, brush allows abiotics changes, ability to toggle species to show and unshow, ability to undo and redo changes, ability to export to a .pdb file
Validation Execution of all user interactive features were validating through extensive testing of the GUI and all file paths.

Output handling

Requirement: The user must have the ability to output a .pdb file at any time in order to save their progress The format is that of a .pdb file that the user can then use to upload to either EcoViz or our in-built 3d generator. *Validation:* The 3D generator utilizes a .pdb file that output a representation generated by the program, validating that its output must be accurate as it matches the 2D representation.

5. Conclusion

Programming an interactive simulation of an ecological environment is a daunting undertaking, and we feel satisfied with our implementation of the requirements. The program effectively models a realistic plant ecosystem, correctly performing calculations based on scientifically-backed algorithms. The functionality for the user to interact with and update the ecosystem snapshot provides a fantastic user experience and has the ability to help researchers and ecologists in their work. The system design is also modular in a way that would allow seamless adding of new features in the future and makes enhancements to existing classes easy to make. This object-oriented design approach makes testing of different logic components simple and makes accessing the 2D arrays and object properties simple for all methods across the program. Caching of some values provided good optimization and the user experience is impressive considering the complication of the computations and data structures. The brush tool and ability to change the parameters of a species is an impressive feature for users, and the user experience is optimized by allowing the undo and redo of changes. The ability to only view certain species is also helpful for viewing insights of changes without clutter. Producing a .pbd file and then being able to render a 3D representation of it satisfies the extension requirement well and further improves the user experience.

Programming an interactive simulation of an ecological environment is a daunting undertaking, and we feel satisfied with our implementation of the requirements. The program effectively models a realistic plant ecosystem, correctly performing calculations based on scientifically-backed algorithms. The functionality for the user to interact with and update the ecosystem snapshot provides a fantastic user experience and has the ability to help researchers and ecologists in their work. The system properly integrates abiotic data input, pink noise sampling, plant placement based on viabilities, and height calculation into a scientifically based eco simulation. The results of the brush and global updates show that the generated forests respond intuitively to underlying environmental conditions, with both 2D and 3D visualizations modelling the data correctly in a way that is visually confident with real life ecosystems. The performance goals were achieved to a level we are satisfied with, and the slope and colour precalculation and caching reduced GUI delays. It also allowed the brush tool to apply local changes responsively for the larger grid sizes. While initial loading and very large simulations still could still benefit from speed up, the overall experience was smooth and scalable for the most useful and common use cases. The GUI was successfully able to allow users to manipulate environmental factors and species parameters and the undo/redo and selective species viewing improved customization and usability. Our extensive testing also confirmed that all the functional requirements were fulfilled. The program therefore achieves both its accurate and sophisticated eco modelling aims and its practical goals for responsiveness and usability. We are also satisfied that we went above and beyond the requirements in providing the most informative visualization of the multiple abiotic maps with the forest to the users.

6. User Manual

Please see user manual under README.md in the submitted folder.

References

A., Gain, J., Guerin, E., Argudo, O. and Galin, E., "DeadWood: Including Disturbance and Decay in the Depiction of Digital Nature", ACM Transactions on Graphics, 2024, Vol. 43, No. 2, pp. 21:1-19.

K., Gain, J., Guérin, E., Galin, E., Peytavie, A., "Datadriven Authoring of Large-scale Ecosystems", ACM Transactions on Graphics, December 2020, Vol. 39, No. 6.

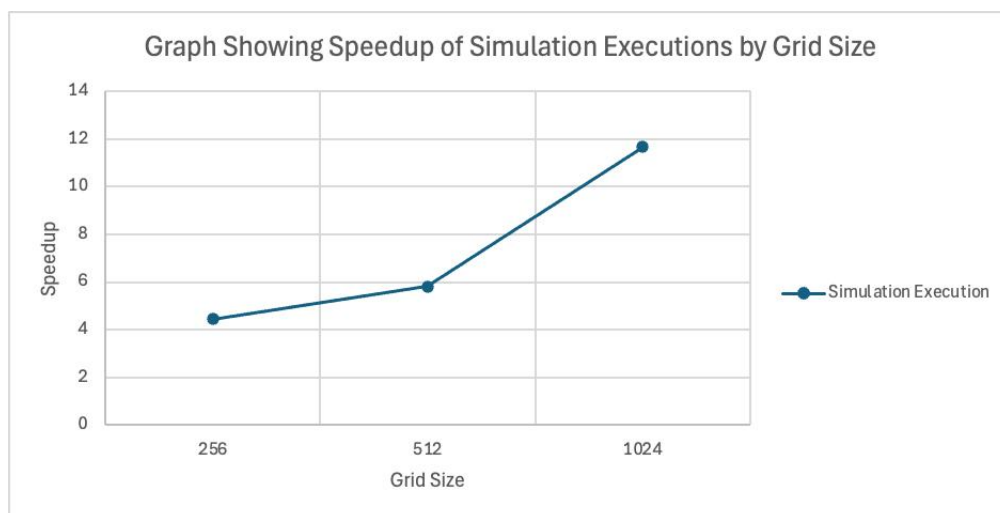


Figure 5: Speedup Diagram for our Application